

RAG Knowledge-Base System — Setup & Usage Guide

How to stand this system up from scratch, put it to work with real documents, and run it day to day. Written for whoever is setting this up next — including someone who wasn't part of building it.

What This Is

This is a self-hosted AI research assistant with its own private knowledge base. You feed it documents (PDFs, Word files, PowerPoint decks, photos of signs or placards, database tables) and it answers questions by actually searching that material first, citing where each answer came from, rather than guessing from general knowledge. It runs entirely on your own small server — nothing about your documents or questions goes to a third-party RAG service, though the AI model calls themselves (chat and embeddings) go through OpenAI's API.

Three pieces work together: Postgres (a database, storing your content as searchable numeric vectors), LiteLLM (a single gateway that every AI model call passes through), and Hermes (the agent that actually talks to you, decides when to search the knowledge base, and can also browse the live web when your documents don't have an answer).

What You'll Need

- **A small VPS.** This was built and tested on 3.8GB RAM / 2 vCPU / 28GB disk (a genuinely tight box) — that's a reasonable minimum, not a recommendation to go smaller. Ubuntu 22.04 or similar.
- **An OpenAI API key.** Required — this is what powers chat and embeddings by default. A Gemini and/or Groq key are optional, only needed if you want those as additional model options.
- **Basic command-line comfort.** Setup is done over SSH. The automated `setup.sh` script handles most of it, but you'll still need to SSH in, edit one file, and run one command.

Setup, Step by Step

The fast path is the included **`setup.sh`** script, which automates everything below except creating your server and filling in your API keys. Run it, and it will tell you exactly what's missing at each step. The manual steps are documented here too, in case you need to troubleshoot or understand what it's doing.

1. Provision the server and install Docker

Any Ubuntu VPS from a standard cloud provider works. `setup.sh` installs Docker and Docker Compose automatically if they're not already present.

2. Get the files onto the server

Clone or copy the `hermes-rag-pipeline` repo's contents onto the server. The `hermes-stack/` folder needs to end up at `~/hermes-stack` on the server (`docker-compose.yml`, `litellm/config.yaml`, and the `init-scripts/` folder all live there), with the pipeline scripts (`rag_ingest.py`, `rag_query.py`, `SOUL.md`, and the two regression-test files) available one level up, next to `hermes-stack/`.

3. Configure secrets

Copy `.env.example` to `.env` inside `hermes-stack/` and fill in real values — at minimum a Postgres password you choose yourself, and your OpenAI API key. Never commit the real `.env` file anywhere.

```
cd ~/hermes-stack
cp .env.example .env
nano .env # fill in POSTGRES_PASSWORD and OPENAI_API_KEY at minimum
```

4. Run setup.sh

This starts all three containers, waits for Postgres to come up healthy, creates the two database tables the pipeline needs (rag_chunks for the knowledge base itself, rag_query_cache for repeated-question caching) along with their search indexes, and installs the pipeline scripts into a Hermes profile of your choosing (researcher by default).

```
chmod +x setup.sh
./setup.sh
```

It's safe to re-run — every step checks whether it's already done before acting.

5. Verify

```
docker compose -f ~/hermes-stack/docker-compose.yml ps
```

You should see postgres (healthy), litellm, and hermes all Up.

Using the System

Getting content in. Everything goes through rag_ingest.py, run inside the Hermes container. Four modes are supported:

- **wiki** — pulls and stores a Wikipedia article by title.
- **file** — ingests a PDF, Word doc, PowerPoint, text file, or photo already sitting on the server. PDFs get page-by-page table detection; scanned pages and images get OCR'd automatically.
- **db** — pulls rows from a database table where one column holds the relevant free text.
- **dbrow** — pulls rows from a table where the useful information is spread across several columns (e.g. a product or equipment table) — every selected column gets included.

```
docker exec hermes python3 /opt/data/rag_ingest.py file /opt/data/some-document.pdf
docker exec hermes python3 /opt/data/rag_ingest.py wiki "Article Title"
docker exec hermes python3 /opt/data/rag_ingest.py dbrow "SELECT name, price FROM products"
products
```

Asking questions. rag_query.py takes a question and an optional source filter, and returns the best-matching stored content along with two important signals printed right after the question:

- **[retrieval quality: STRONG/WEAK/NONE]** — whether the knowledge base actually has a confident answer. Hermes is instructed to treat WEAK or NONE as "don't force an answer from this."
- **[conflict detected: ...]** — appears only when two or more of your own documents disagree on a fact (different prices, different dates). Hermes is instructed to report both sides rather than silently pick one.

```
docker exec hermes python3 /opt/data/rag_query.py "your question here" optional_source_filter
```

In normal use, you won't run either script directly — Hermes calls them itself as part of answering your questions, deciding on its own when a knowledge-base check makes sense versus a live web search. The commands above are for testing and troubleshooting.

Checking nothing's broken. run_regression_tests.py replays a saved set of known question/answer pairs against the live pipeline and reports pass/fail — run this after any change to the scripts or server config,

before trusting it with real use again.

```
docker exec hermes python3 /opt/data/run_regression_tests.py
```

Customizing for a New Business or Use Case

Everything business-specific lives in one file: SOUL.md, inside the profile's folder (hermes-stack/data/hermes/profiles/<profile-name>/SOUL.md). This is where you describe what the agent's job is, its tone, and when it should check the knowledge base versus search the web or fall back to general knowledge. The pipeline scripts themselves (rag_ingest.py, rag_query.py) don't need to change between businesses — only what gets ingested and how SOUL.md instructs the agent to use it.

To stand up a second, separate profile (a different business, or a different role for the same business), create a new folder under profiles/, copy in SOUL.md as a starting point and rewrite it, then re-run the relevant part of setup.sh (or copy the scripts in manually) pointed at the new profile name.

Ongoing Maintenance

- **Restarting Hermes** after it's been stopped: docker compose start hermes (from hermes-stack/). Postgres and LiteLLM can be left running continuously.
- **Dependencies surviving restarts:** a container restart wipes anything installed outside the persistent data volume. init-scripts/04-rag-deps reinstalls the OCR and document-parsing tools automatically every time the container starts — don't remove or skip this script.
- **The query cache** (rag_query_cache) cleans itself up automatically — entries expire after 60 minutes and get deleted outright after 24 hours, so it never needs manual attention.
- **Model costs:** classification and per-chunk context generation run on gpt-5-nano (cheap). Reranking runs on gpt-5-mini — this was deliberately reverted from nano after nano proved unreliable at the reranker's combined relevance/quality/conflict judgment (see the build summary, Section 17, for why).

Troubleshooting / Known Quirks

- **Hermes's own terminal tool mishandles apostrophes** in a question passed to rag_query.py — a question containing "what's" or similar can silently return zero results even though the same question run directly works fine. This lives inside the packaged Hermes agent, not in these scripts. Rephrasing without the apostrophe works around it.
- **AI calls with a low max_tokens setting can silently return empty content** on reasoning models (gpt-5-nano, gpt-5-mini) — they spend part of their token budget on hidden reasoning before writing a visible answer. If you add a new AI-calling function, give it a generous budget (500-1500) and test with realistic input sizes before trusting it.
- **PDF table extraction and OCR both depend on the boot script** (04-rag-deps) having actually run. If table extraction or scanned-document ingestion suddenly stops working after a server-level change, check that this script is still wired into docker-compose.yml.
- **Memory is tight.** This was built against a 3.8GB RAM server. Don't add heavier libraries (the unstructured document-parsing library was tried and reverted for exactly this reason — see the build summary, Section 6 and Section 9) without confirming the server can actually hold them.